
BIASMIX-FINANCE: POST-GENERATION KYC GUARDRAILS FOR LLM PORTFOLIO ADVICE

Gaurav Kukreja
gauravkukreja06@yahoo.in

Parul Kukreja
parulsehgal11@gmail.com

Mohammed Abraar
abraar@vizz.vizuara.ai

Raj Dandekar
raj@vizuara.com

Rajat Dandekar
rajatdandekar@vizuara.com

Sreedath Panat
sreedath@vizuara.com

ABSTRACT

Large language models (LLMs) can generate plausible-sounding ETF portfolios while silently violating basic KYC-style constraints on risk, fees, and diversification. This is especially problematic in *agentic* multi-turn advisory systems, where each draft recommendation can become an action unless guarded by an auditable enforcement layer. We study a model-agnostic, asset-agnostic *post-generation guardrail* pipeline: (i) enforce a strict JSON allocation schema, (ii) validate allocations against numeric caps, and (iii) when violations occur, *deterministically* project the output to the nearest feasible portfolio via a convex quadratic program (QCQP). We introduce **BiasMix-Finance (Mini)**, a compact stress-test benchmark for *constrained decision-making under biased LLM generations*, with a 16-ETF universe, three investor profiles, and eight bias prompts. Across three models and three inference modes (direct, critique, self-consistency), first-pass generations violate at least one cap in **47.6–85.7%** of test cases (**67.2%** pooled), but the convex projection layer reduces *final feasibility violations to 0%* while requiring only a *small correction distance* (test pooled median $D = \|w^* - w_0\|_2 = 0.066$), indicating that the guardrail typically preserves the intent of the original allocation. We report violation rates and correction distances with confidence intervals, and paired model comparisons with multiple-testing correction. To support reproducibility, we release the dataset, prompts, caps, and code in our public GitHub repository.

1 Introduction

Large language models (LLMs) are increasingly used to generate financial summaries, recommendations, and portfolio allocations [33, 34]. However, in regulated advisory settings, recommendations are expected to reflect customer-specific facts and suitability obligations [12, 13]. In our controlled setting, we operationalize this idea as hard numeric caps on risk, fees, and diversification, which are also standard in constrained portfolio construction [6, 19, 21]. Prompting and reasoning-mode variations (e.g., critique, self-consistency) are inherently probabilistic and cannot guarantee constraint satisfaction on every run [20, 28, 29]. This motivates **post-generation enforcement**: we treat the LLM output as a draft allocation and apply deterministic verification and repair, combining audit-style validation with convex feasibility projection [6, 24, 26]. Crucially, our goal is *not* to solve a classical portfolio-optimization problem (e.g., maximizing risk-adjusted return under constraints) [5, 14, 21]. Instead, we study an *auditing and repair* problem: how to reliably detect and minimally correct *irrational or constraint-violating* allocations produced by an LLM, while

preserving the model’s intended allocation structure as much as possible [3, 24, 26]. More broadly, this setting is adjacent to LLM alignment and policy-following work [1, 23], but differs because correctness depends on satisfying externally specified constraints that can be verified by deterministic checks. Because these constraints are externally specified and mechanically checkable, the task is a natural testbed for *post-hoc verification and repair* pipelines that are model-agnostic and transferable to other constrained decision-making domains (e.g., resource allocation, scheduling, or policy compliance). We study a controlled portfolio-allocation setting: given an ETF universe and a scenario describing (i) a risk profile with caps and (ii) a bias-inducing context (e.g., “small-cap hype”), the LLM emits portfolio weights under a strict JSON schema. A key challenge is that biased or preference-shaping contexts can systematically push generations toward concentrated, high-volatility, or otherwise non-compliant allocations. To evaluate guardrails under these realistic failure modes, we adopt a **bias-induction methodology**: we programmatically generate scenarios that combine standardized risk profiles with bias “recipes” (e.g., anchoring on a sector, FOMO tilts, inertia, fee neglect), producing a stress-test suite for constraint-violating drafts. We then measure constraint violations *before* repair and the magnitude of correction required *after* repair. Our goal is to quantify how much prompt reasoning modes reduce violations *prior* to enforcement, and how much post-hoc guardrails must change the portfolio to make it compliant. In this sense, we treat BiasMix-Finance (Mini) as a stress-test benchmark for *constrained decision-making under biased generations*, complementing broader evaluation efforts in safe and reliable LLM deployment. To verify that BiasMix prompts elicit the intended behavioral tilts, Appendix Table 6 shows representative first-pass LLM outputs (held-out test) for each bias recipe. We focus on post-generation enforcement, which is especially relevant for *agentic* financial advisors that operate in multi-turn loops and tool-call to validate actions. Section 7 discusses how deterministic verify-and-repair acts as a governance mechanism inside autonomous advisor systems.

Contributions. We emphasize that the contribution is a *methodology and evaluation framework* for deploying LLMs in constrained financial decision workflows, rather than a new optimization technique, the QCQP projection is used as a deterministic repair operator.

- **Treating LLM outputs as auditable drafts:** We treat LLM-generated allocations as *auditable drafts* that must satisfy explicitly specified, mechanically checkable KYC-style numeric caps, enabling transparent logging of (draft, violations, repaired output) for governance and oversight.
- **Bias-induced stress testing as an evaluation primitive:** We construct 72 scenario instances (train/dev/test) that combine three risk profiles with eight behavioral “bias recipes” and three random seeds, over a fixed 16-ETF universe. This controlled design isolates how different prompts and models respond to the same constraints and bias contexts, and makes bias induction itself a reproducible evaluation primitive for constrained decision-making under LLM outputs.
- **Deterministic repair as a first-class safety mechanism:** We formalize the feasible set induced by volatility, fee, HHI concentration, and max single-asset/sector caps, and compute the nearest feasible portfolio via a constrained quadratic program, solved with standard convex optimization tools [6, 8, 22, 27].
- **Reproducible statistical evaluation across splits, models, and modes:** We report per-cap violation rates with Wilson 95% confidence intervals [32], correction-distance and metric-delta confidence intervals via bootstrap [11], and paired model comparisons on correction distance with Wilcoxon tests plus BH-FDR correction [4, 30].

2 Problem Setup

Universe. Let n denote the number of ETFs in the universe (here $n = 16$) and let $w \in \mathbb{R}^n$ denote portfolio weights.

Scenario. Each scenario provides (i) a risk profile with caps and (ii) a bias context string intended to nudge the model toward potentially non-compliant allocations. Caps include an annualized volatility ceiling σ_{cap} , a weighted-average expense ratio ceiling f_{cap} , a concentration ceiling h_{cap} based on the Herfindahl–Hirschman Index (HHI) [15, 16], and max single-asset and max single-sector weight caps. Such constraints are standard in constrained portfolio construction and are commonly studied as practical overlays on mean–variance style allocations [19, 21].

Model output. The LLM emits a draft portfolio w_0 (JSON with nonnegative weights that sum to 1). We compute summary metrics: risk $\sigma(w) = \sqrt{w^\top \Sigma w}$ [21], fee $\text{WAER}(w) = \sum_i w_i \cdot \text{fee}_i$, and concentration $\text{HHI}(w) = \sum_i w_i^2$.

Feasible set. Let $\mathcal{C}$ be the set of portfolios satisfying all caps. A draft is *violating* if $w_0 \notin \mathcal{C}$.

Post-generation enforcement. When w_0 violates any cap, we compute the nearest feasible portfolio

$$w^* = \arg \min_w \|w - w_0\|_2^2 \quad \text{s.t.} \quad w \in \mathcal{C}, \quad (1)$$

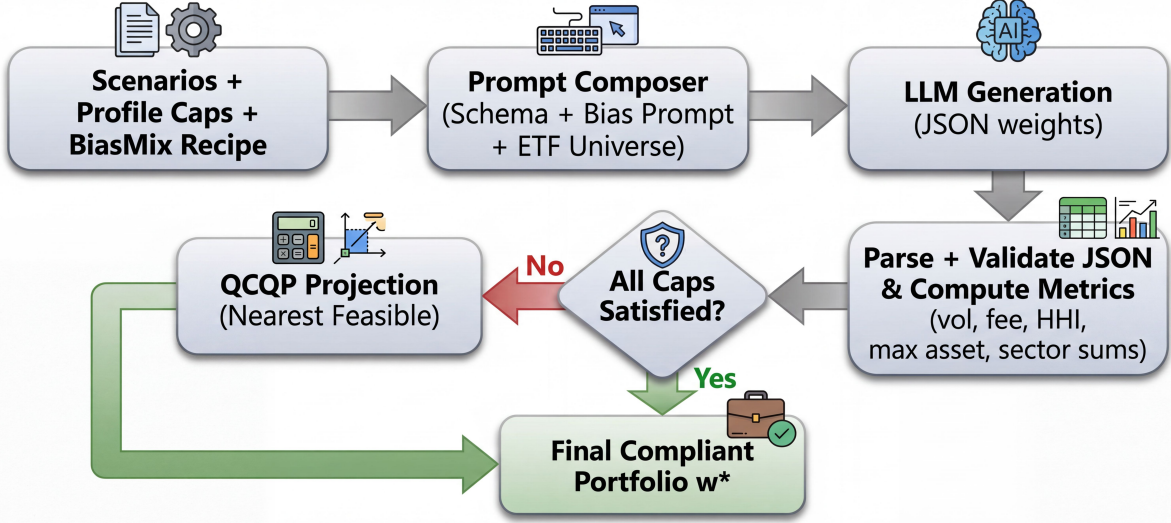


Figure 1: **End-to-end pipeline.** BiasMix scenario + bias recipe are composed into a structured prompt. The LLM generates draft weights (JSON). We validate JSON and compute portfolio metrics against hard KYC caps. If any constraint fails, we repair via convex projection (QCQP) to the nearest feasible portfolio; otherwise we accept directly.

and define the correction distance as $D = \|w^* - w_0\|_2$ (Euclidean/L2 distance). Projection operators are a standard way to enforce feasibility under convex constraints [6, 10], and related portfolio work often uses robust or constrained formulations to stabilize allocations under estimation error [2, 5, 14].

3 Related Work

LLMs for finance. Domain-specific financial LLMs such as BloombergGPT [33] and open efforts like FinGPT [34] show strong capability on financial NLP tasks; our focus differs in that we study *hard numeric constraint compliance* for portfolio-weight outputs rather than text generation accuracy.

Guardrails, auditing, and constrained outputs. Safety toolkits and classifiers (e.g., NeMo Guardrails, Llama Guard) [18, 25] and alignment methods such as RLHF and Constitutional AI [1, 23] primarily target content/policy enforcement. Complementary work emphasizes *auditing* and *verification* of LLM outputs in high-stakes settings via structured outputs, validation checks, and feedback loops [24, 26], and constrained/guided decoding provides guarantees for lexical or structural constraints [17, 31]. Our setting requires satisfying quantitative constraints naturally expressed as convex restrictions over portfolio weights; this motivates optimization-based post-processing as an auditable repair layer [6]. We discuss additional related work and context in Appendix H.

4 Methodology

Figure 1 summarizes the end-to-end BiasMix-Finance pipeline from scenario construction to deterministic feasibility via projection [6, 26].

4.1 Universe and scenarios

We construct a synthetic-but-controlled scenario suite over a fixed ETF universe of $n = 16$ liquid funds: SPY, VEA, VWO, VGT, XLE, XLF, XLV, XLY, XLP, XLI, XLRE, IWM, AGG, LQD, IEF, GLD. Each scenario specifies an *as-of* date (we use 2025-09-30), a risk profile (Conservative/Moderate/Aggressive), and a *BiasMix* recipe that nudges the model toward a known behavioral bias (e.g., “anchor tech”, “FOMO energy”, “small-cap hype”). Recipes are expressed as short natural-language preferences (not hard constraints) so that first-pass outputs can be either feasible or infeasible. This design parallels behavioral stress testing in NLP evaluation: the goal is to systematically elicit failure modes under controlled perturbations [26]. We generate 72 scenarios across 3 profiles and 8 bias recipes, with *three* random seeds per (profile, recipe) cell; split sizes are train/dev/test = 36/15/21, balanced across the three risk profiles (each profile contributes 12/5/7 to train/dev/test). We use train only for prompt and hyperparameter iteration (e.g., mode

Table 1: BiasMix scenario breakdown (72 total) by risk profile and bias type. Each (profile, bias) cell has 3 scenarios (3 random seeds).

Profile	Anchor tech	Default inertia	EM tilt	FOMO energy	Fee neglect	Gold craze	Small-cap hype	US-only bias	Total
Conservative	3	3	3	3	3	3	3	3	24
Moderate	3	3	3	3	3	3	3	3	24
Aggressive	3	3	3	3	3	3	3	3	24
Total	9	9	9	9	9	9	9	9	72

Table 2: Risk-profile caps used throughout. σ_{cap} is annualized volatility, WAER is weighted-average expense ratio, HHI is concentration, a_{max} is max single-asset weight, and s_{max} is max single-sector weight.

Profile	σ_{cap}	f_{cap} (WAER)	h_{cap} (HHI)	a_{max}	s_{max}
Conservative	0.06	0.0020	0.12	0.25	0.30
Moderate	0.10	0.0030	0.18	0.35	0.40
Aggressive	0.14	0.0040	0.25	0.45	0.50

temperatures, self-consistency sample size K , and projection margins). Dev a one-time check that the tuned pipeline behaves similarly on unseen data. We report results on test without further tuning. Table 1 summarizes the dataset. We use train/dev/test splits only to avoid leakage from repeated prompt/solver tuning; split construction details are in Appendix B.5.

4.2 Risk profiles and hard caps

All caps are defined over the portfolio functions in Section 2. For each risk profile we fix a cap vector $\theta = (\sigma_{\text{cap}}, f_{\text{cap}}, h_{\text{cap}}, a_{\text{max}}, s_{\text{max}})$ covering annualized volatility, weighted-average expense ratio (WAER), concentration (HHI), and single-asset / single-sector limits. Table 2 lists the numerical values used in all experiments.

Cap rationale The caps in Table 2 are chosen as representative KYC-style suitability heuristics inspired by customer-specific suitability and diversification principles, not jurisdiction-specific rules [12, 13, 19]; detailed justification and threshold sensitivity are in Appendix B.1.

4.3 Guardrail pipeline

Given a scenario, the model outputs portfolio weights w_0 under a strict JSON schema. We validate w_0 ; if any constraint in $\mathcal{C}(\theta)$ is violated, we compute a nearest-feasible projection:

$$w^* = \arg \min_{w \in \mathcal{C}(\theta)} \|w - w_0\|_2^2 + \lambda w^\top \Sigma w, \quad (2)$$

where feasible set $\mathcal{C}$ is induced by the hard caps:

$$\mathcal{C} = \left\{ w \in \mathbb{R}^n : \begin{array}{l} \mathbf{1}^\top w = 1, w \geq 0, \\ \sigma(w) \leq \sigma_{\text{cap}}, \text{ WAER}(w) \leq f_{\text{cap}}, \\ \text{HHI}(w) \leq h_{\text{cap}}, \max_i w_i \leq a_{\text{cap}}, \\ \text{sector_sum}(w) \leq s_{\text{cap}} \end{array} \right\} \quad (3)$$

$\lambda = 2 \times 10^{-3}$ is a small variance regularizer that discourages solutions sitting exactly on the risk boundary. We report the *correction distance* $D = \|w^* - w_0\|_2$ (Euclidean norm). We solve the projection as a convex QCQP with standard solvers; numerical stability choices (PSD jitter), solver tolerances, fallback strategy, and robustness ablations are reported in the Appendix D

4.4 Models and providers

We evaluate three LLM backends: (i) *Gemini 2.5 Flash* (gemini-2.5-flash) via the Google GenAI Python SDK, (ii) *GPT-5 nano* (gpt-5-nano) via the OpenAI Chat Completions API, and (iii) an open-weight model served through an OpenAI-compatible endpoint, *Llama 3.3 70B Instruct Turbo* (meta-llama/Llama-3.3-70B-Instruct-Turbo).

4.5 Inference Strategies (Modes)

The unit of analysis is a *scenario* evaluated under a (model, mode) configuration. We evaluate the models across three strategies: (i) **Direct** generation, where the model is prompted to produce the target JSON output in a single shot; (ii) **Critique**, a multi-turn approach where the model generates an internal draft and critique before emitting a corrected JSON [20]; and (iii) **Self-consistency (SC)**, where $K = 5$ candidates are sampled, and the candidate with the lowest pre-projection violation is selected [28]. All modes share the same post-generation projection step when constraints are violated and enforce strict JSON outputs with up to $R=3$ retries on parse failure; once parsed, weights are validated and projected if needed. Full decoding and parsing details are in the Appendix E.

5 Experimental protocol and statistical framework

We evaluate each scenario under a (model, mode) configuration and measure compliance on the first-pass draft w_0 and after deterministic repair w^* . Our **primary endpoints** are (i) first-pass violation rate (per-cap and any-cap) and (ii) correction distance $D = \|w^* - w_0\|_2$. We additionally report parse-failure rate and before/after deltas in σ , WAER, and HHI. For violation rates we use Wilson 95% CIs [32]; for D and deltas we use bootstrap 95% CIs [11]. For paired model comparisons on D we use Wilcoxon signed-rank tests with BH-FDR correction [4, 7, 30]. Full protocol details (splits, resampling, and multiple-testing family) are in Appendix F.

6 Results

6.1 First-pass violations on held-out test

Table 4 and Figure 2 report pooled first-pass violation rates on the test split. Critique and self-consistency tend to reduce violations relative to direct prompting, but do not guarantee compliance. Violation rates are heterogeneous across BiasMix recipes and profiles; Appendix Figs. 4–6 visualize test-split violation heatmaps by bias type, profile, and model (for each prompting mode). Interpreting the magnitudes, prompting alone remains unreliable under hard KYC-style caps: even the best test setting (Gemini+SC) violates in nearly half of cases (0.476), while the worst (Gemini+direct) violates most of the time (0.857). Self-consistency helps most when the feasible region is larger (Aggressive drops from 0.476 to 0.048), but tight regimes remain difficult: Conservative portfolios violate on all test scenarios across models/modes (1.0), motivating a deterministic verify-and-repair layer. The Conservative regime is intentionally *extremely tight*: the intersection of a low volatility cap, a strict HHI concentration cap, and sector/asset limits induces a feasible region that is very small on the simplex. In practice, this means the admissible set resembles a narrow polytope/conic slice that occupies a tiny fraction of the space of plausible-looking portfolios. Because prompting and reasoning modes remain probabilistic, they cannot reliably “hit” such a small feasible region in one shot, even when the generated portfolio appears sensible at a glance. This is precisely the setting where deterministic post-generation repair is necessary: it converts an intent-bearing but infeasible draft into a compliant action while minimizing deviation from the draft, making hard-cap satisfaction a property of the system interface rather than the sampling behavior of the model.

6.2 Projection yields high final feasibility

Most failures arise at parse time (invalid JSON), not at constraint time. On the held-out test split, with strict schema prompting and up to $R = 3$ retries, we observe *zero parse failures* and *100% post-projection feasibility* across all models and modes (Wilson 95% CIs: parse-fail $[0, 0.155]$, final pass $[0.845, 1.0]$; full table in Appendix C.2). Thus, reliability hinges primarily on structured output; once weights are parsed, solver-based repair enforces constraints deterministically while preserving the LLM as an intent generator.

6.3 How much correction is required?

Table 3 quantify the adjustment required to reach feasibility, the corresponding visualization is in Appendix C.4. Lower D indicates the model proposed a near-feasible portfolio. We interpret D as a faithfulness signal: small D means projection only nudges the draft onto the feasible set, while large D means constraints substantially override the draft. Consistent with tightness, corrections are smallest for Aggressive (often near zero, especially with SC), moderate for Moderate (roughly 0.018–0.185), and largest for Conservative (roughly 0.220–0.462), aligning with its near-certain first-pass violations. In deployment, large D can be surfaced as “constraint-forced rebalancing,” while small D supports the claim that guardrails can guarantee compliance without materially changing near-feasible drafts. Beyond L2 distance, intent is also preserved at a coarser granularity: Appendix Figs. 7–9 plot draft vs. projected *sector* allocations on the

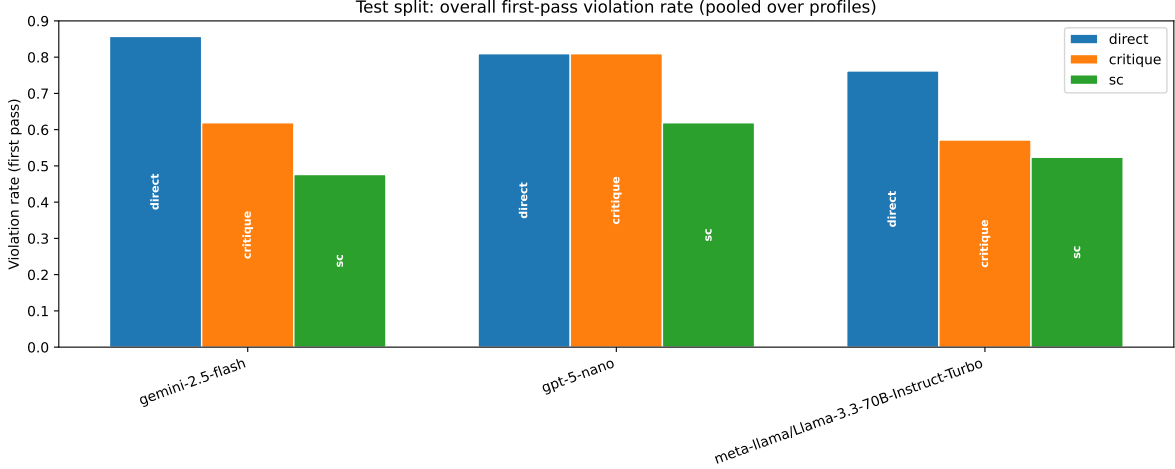


Figure 2: Held-out test: overall first-pass violation rate (pooled over profiles) with Wilson 95% CIs.

Table 3: **Test correction distance D (bootstrap 95% CI).** $D = \|w^* - w_0\|_2$ (unitless L2 distance on the simplex); lower values mean the model output was closer to feasible before projection.

Model	Mode	n	D mean (band)	D median
gemini-2.5-flash	critique	21	0.108 [0.000,0.383]	0.028
gemini-2.5-flash	direct	21	0.186 [0.009,0.591]	0.048
gemini-2.5-flash	sc	21	0.104 [0.000,0.376]	0.000
gpt-5-nano	critique	21	0.127 [0.007,0.320]	0.121
gpt-5-nano	direct	21	0.203 [0.012,0.435]	0.110
gpt-5-nano	sc	21	0.097 [0.000,0.276]	0.072
Llama-3.3-70B-Instruct	critique	21	0.106 [0.000,0.331]	0.029
Llama-3.3-70B-Instruct	direct	21	0.136 [0.000,0.358]	0.119
Llama-3.3-70B-Instruct	sc	21	0.097 [0.000,0.322]	0.028

test split, with points concentrated near the diagonal. On the held-out test split (pairs=21), differences are significant in *direct* and *critique* but not *SC*: gpt-5-nano vs meta-llama/Llama-3.3-70B-Instruct-Turbo is significant in *direct* ($p_{\text{FDR}}=0.0292$) and *critique* ($p_{\text{FDR}}=0.0098$), and Gemini-2.5-flash vs meta-llama/Llama-3.3-70B-Instruct-Turbo is also significant in *direct* ($p_{\text{FDR}}=0.0669$) and *critique* ($p_{\text{FDR}}=0.0098$). Full results across splits and modes are in Appendix 7.

7 Implications for Agentic Financial AI

FinAI emphasizes *agentic* financial systems: autonomous or semi-autonomous advisors that interact over multiple turns, call tools, and iteratively refine decisions under constraints. Although BiasMix-Finance is evaluated as a post-generation guardrail for a single draft, the same verify-and-repair interface naturally composes with agent loops. In this view, the LLM is an *intent generator* (proposal policy) and the guardrail is an *enforcement tool* that ensures every externally visible action satisfies hard KYC-style constraints.

Multi-turn advisor interactions. As users update preferences or facts (e.g., fee sensitivity, horizon, concentration tolerance), a multi-turn agent can treat each intermediate allocation as a tentative action, run validation and projection, and return a feasible portfolio at *every turn*. Even if intermediate reasoning is noisy, deployed outputs remain within a clearly specified feasible region. The correction distance $D = \|w^* - w_0\|_2$ can be surfaced as an explanation signal for how strongly constraints overruled the draft (“light nudge” vs. “constraint-forced rebalancing”).

Iterative correction loops in autonomous agents. Agentic systems often operate as repeated propose–check–revise cycles [20, 35]. Our pipeline provides a deterministic inner loop: propose an allocation, validate, project to feasibility,

and feed the corrected portfolio back into the agent state. This replaces brittle prompt-only self-correction with a tool-grounded mechanism that enforces admissible actions whenever projection succeeds.

Guardrails as governance mechanisms. In regulated settings, governance requires policies that are auditable, stable, and configurable (e.g., per institution, jurisdiction, or product type). Numeric caps and deterministic repair provide a concrete governance layer: constraints are explicit and versionable, and every recommendation can be logged with (draft, violations, repair, and D).

Feedback adaptation from corrected feasible outputs. The (draft $\rightarrow$ projected) pairs produced by the guardrail are high-signal supervision: they encode how a recommendation must change to satisfy policy. This supports (1) *prompt-level adaptation* that conditions on prior corrections to draft closer-to-feasible portfolios (reducing D), and (2) *model-level adaptation* using corrected outputs for fine-tuning or preference learning, while retaining repair as a backstop.

Regulatory sandboxing for agent-based financial advisors. BiasMix-Finance can serve as a lightweight *regulatory sandbox* by simulating multi-turn dialogues: vary the investor profile, inject biased or adversarial instructions, and measure (i) first-pass violation propensity, (ii) correction distance D , and (iii) stability of sector/asset allocations after repair. Because the enforcement layer is deterministic and policy-driven, the same sandbox can be repeated under different cap configurations to stress-test agent behavior as governance tightens or loosens.

Overall, the contribution is not only a reliable post-processor for static drafts, but a composable *agent tool* for financial decision-making: it turns probabilistic proposals into policy-compliant actions for safer multi-turn and autonomous advisory workflows.

8 Discussion, limitations, and future work

Prompting modes (direct, critique, self-consistency) can reduce first-pass violations, but they cannot guarantee satisfaction of hard numeric caps because generation remains probabilistic. In contrast, the post-generation projection step provides an auditable, model-agnostic enforcement layer that deterministically returns a feasible portfolio (when the cap set is feasible), while minimizing deviation from the model’s proposal.

Scope and limitations. This study is intentionally scoped to *constraint compliance and robustness*, not portfolio optimality: we test whether LLM allocations can be made KYC-feasible under explicit caps and quantify the minimal adjustment required (correction distance D). We therefore do not model expected returns, transaction costs, taxes, or utility-based objectives. We use a fixed *16-ETF* universe as a controlled stress test; the guardrail is not tied to 16 assets and applies to any universe with fees, sector mappings, and a covariance estimate. We also do not conduct real-user or advisor-in-the-loop studies, focusing instead on an auditable enforcement mechanism for downstream workflows.

Additional discussion on universe design and generalization is provided in Appendix G.

Transferability beyond portfolio allocation. The verify and repair pattern studied here applies to a broader class of *agentic financial decision systems* where an LLM proposes a structured action but correctness depends on hard, externally specified constraints. Examples include budget allocation under category caps, product eligibility and suitability checks, policy-compliant trade lists (e.g., prohibited assets, minimum lot sizes, concentration and exposure limits), disclosure and fee-rule compliance, and structured credit/insurance recommendation drafts that must satisfy underwriting or regulatory rules. Across these settings, the key systems principle is to separate *proposal generation* (probabilistic, model-driven) from *admissibility enforcement* (deterministic, auditable). BiasMix-style stress testing also transfers by inducing known behavioral pressures (e.g., fee neglect, overconfidence, anchoring, recency) and measuring (i) first-pass violation propensity and (ii) the magnitude of repair required to restore compliance. Thus, the primary contribution is a reusable governance-oriented framework: explicit constraints, deterministic repair, and measurable faithfulness between draft and compliant action.

Future work. Promising extensions include (i) scaling to larger universes and adding liquidity/turnover constraints, (ii) replacing the fixed covariance with rolling or regime-aware estimates, (iii) incorporating return-aware or utility-based objectives while preserving hard caps, and (iv) running human-evaluation studies to assess whether corrected portfolios preserve user intent and improve trust and usability.

9 Conclusion

Post-generation KYC-style guardrails offer a practical and auditable way to enforce numeric investment constraints on top of LLM-generated allocations. Across models and prompting modes, we find that a convex nearest-feasible projection can eliminate final constraint violations while keeping repairs bounded in L2 distance. More generally, the same verify-and-repair pattern is *model-agnostic* and *asset-agnostic*: any LLM that emits a structured decision can be checked against hard constraints and deterministically repaired when needed, making the approach applicable beyond finance. BiasMix-Finance (Mini) serves as a stress-test benchmark for constrained decision-making under bias, enabling controlled comparisons of reasoning modes and models under identical caps and contexts.

LLM usage disclosure. We used large language models as auxiliary tools for language polishing and limited code assistance (e.g. code utilities, grammar, and LaTeX formatting). All experimental design, implementation, data collection, analysis, results, and conclusions were produced and verified by the authors, who take full responsibility for the content of this paper.

References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [2] Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust Optimization*. Princeton University Press, 2009.
- [3] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021.
- [4] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- [5] Fischer Black and Robert Litterman. Global portfolio optimization. *Financial Analysts Journal*, 48(5):28–43, 1992.
- [6] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004.
- [7] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [8] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.
- [9] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- [10] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1 -ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 2008.
- [11] Bradley Efron and Robert Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.
- [12] Financial Industry Regulatory Authority. FINRA rule 2090: Know your customer. <https://www.finra.org/rules-guidance/rulebooks/finra-rules/2090>, 2012. Adopted by SR-FINRA-2010-039 and amended by SR-FINRA-2011-016, effective July 9, 2012; accessed: 2026-06-21.
- [13] Financial Industry Regulatory Authority. FINRA rule 2111: Suitability. <https://www.finra.org/rules-guidance/rulebooks/finra-rules/2111>, 2012. Adopted by SR-FINRA-2010-039 and amended by SR-FINRA-2011-016 and SR-FINRA-2012-027, effective July 9, 2012; later amended by SR-FINRA-2020-007, effective June 30, 2020; accessed: 2026-06-21.
- [14] Donald Goldfarb and Garud Iyengar. Robust portfolio selection problems. *Mathematics of Operations Research*, 28(1):1–38, 2003.
- [15] Orris C. Herfindahl. *Concentration in the Steel Industry*. PhD thesis, Columbia University, 1950.
- [16] Albert O. Hirschman. The paternity of an index. *The American Economic Review*, 54(5):761–762, 1964.
- [17] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

- [18] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [19] Ravi Jagannathan and Tongshu Ma. Risk reduction in large portfolios: Why imposing the wrong constraints helps. *The Journal of Finance*, 58(4):1651–1683, 2003.
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- [21] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [22] Brendan O’Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 2016.
- [23] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- [24] Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2020.
- [25] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 431–445, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-demo.40.
- [26] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [27] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12:637–672, 2020.
- [28] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.
- [30] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [31] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- [32] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [33] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- [34] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A Appendix

B Prompts, caps, and scenario template

B.1 Cap rationale and sensitivity

The cap values in Table 2 are intended to be representative of practical “KYC-style” suitability heuristics rather than jurisdiction-specific regulations [12, 13]. They reflect: (i) lower volatility targets for conservative investors; (ii) a low-cost fee budget typical of broad-market ETFs (e.g., 0.20–0.40% WAER); (iii) diversification via concentration control (HHI corresponds to an effective number of holdings of roughly $1/\text{HHI}$); and (iv) concentration limits that prevent over-exposure to a single fund or sector. (We report threshold sensitivity analyses in this appendix.)

B.2 Direct/Critique JSON schema

```
{"weights": {"TICKER": 0.00, "...": 0.00},  
"explanation": "<=120 words"}
```

B.3 SC JSON schema

```
{  
  "candidates": [{"weights": {"TICKER": 0.00, ...}, "explanation": "<=120  
words"},  
  ...  
]  
}
```

B.4 Example BiasMix prompts

We include two representative BiasMix contexts (verbatim) used to induce biased first-pass allocations:

- **Anchor tech:** *Strongly prefer VGT (tech tilt).*
- **Small-cap hype:** *Strongly prefer IWM (small-cap tilt).*

B.5 Train/dev/test splits

Although we do not train the language models, we use splits to avoid *evaluation leakage* from repeated prompt and guardrail tuning on the same scenarios. We stratify scenarios by (profile, recipe) and assign them to **train** (used only to finalize prompts and guardrail solver settings), **dev** (a one-time check that the tuned pipeline behaves similarly on unseen scenarios), and **test** (the final reported results).

Table 4: **Test first-pass violation rate (Wilson 95% CI)**. Violations are measured on the model’s raw proposal w_0 before projection; lower values indicate better prompt-level compliance, not guardrail effectiveness.

Model	Mode	n	Viol.	Violation rate (95% CI)
gemini-2.5-flash	critique	21	13	0.619 [0.409,0.792]
gemini-2.5-flash	direct	21	18	0.857 [0.654,0.950]
gemini-2.5-flash	sc	21	10	0.476 [0.283,0.676]
gpt-5-nano	critique	21	17	0.810 [0.600,0.923]
gpt-5-nano	direct	21	17	0.810 [0.600,0.923]
gpt-5-nano	sc	21	13	0.619 [0.409,0.792]
meta-llama/Llama-3.3-70B-Instruct-Turbo	critique	21	12	0.571 [0.365,0.755]
meta-llama/Llama-3.3-70B-Instruct-Turbo	direct	21	16	0.762 [0.549,0.894]
meta-llama/Llama-3.3-70B-Instruct-Turbo	sc	21	11	0.524 [0.324,0.717]

Table 5: **Test robustness and success rates (Wilson 95% CI)**. Parse-fail counts a scenario only if all $R=3$ retries fail strict JSON parsing. Final feasibility is the post-projection pass rate ($w^* \in \mathcal{C}$). End-to-end success requires both parsing and final feasibility.

Model	Mode	n	Parse fails	Parse-fail (95% CI)	Final pass (95% CI)
gemini-2.5-flash	critique	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
gemini-2.5-flash	direct	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
gemini-2.5-flash	sc	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
gpt-5-nano	critique	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
gpt-5-nano	direct	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
gpt-5-nano	sc	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
Llama-3.3-70B	critique	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
Llama-3.3-70B	direct	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]
Llama-3.3-70B	sc	21	0	0.000 [0.000, 0.155]	1.000 [0.845, 1.000]

B.6 Scenario JSONL template

```
{
  "scenario_id": "AF-001",
  "as_of": "2025-09-30",
  "profile": "Conservative|Moderate|Aggressive",
  "caps": {"sigma_cap":..., "fee_cap":..., "hhi_cap":..., "max_asset":...,
  "max_sector":...},
  "asset_universe": ["SPY","VEA","VWO","VGT","XLE","XLF","XLV",
    "XLY","XLP","XLI","XLRE","IWM","AGG","LQD","IEF","GLD"],
  "biasmix": "<bias recipe text>",
  "seeds": [1],
  "split": "train|dev|test"
}
```

Table 6: **Bias prompt effectiveness on held-out test:** representative first-pass LLM draft allocations and short explanation excerpts for each BiasMix recipe (direct prompting shown for compactness).

Bias recipe	Scenario	Draft weights (excerpt)	Explanation snippet (verbatim)
Anchor tech	AF-051 (Agg.)	VGT 45%, SPY 30%, VEA 15%, VWO 10%	“strong tech tilt via VGT (45%), aligning with the ‘Anchor tech’ bias”
FOMO energy	AF-004 (Cons.)	XLE 10%, AGG 40%, LQD 20%, IEF 17%	“strategic tilt towards XLE (energy) ... aligning with the FOMO energy bias”
Small-cap hype	AF-061 (Agg.)	IWM 25%, SPY 30%, VEA 15%, VWO 10%	“prioritizes ... small-cap exposure via IWM, aligning with the ‘small-cap hype’ bias”
Gold craze	AF-047 (Mod.)	GLD 20%, SPY 20%, AGG 20%, VEA 15%	“a significant gold tilt (GLD) as requested”
EM tilt	AF-060 (Agg.)	VWO 30%, SPY 30%, VEA 20%, VGT 10%	“strong EM tilt via VWO, complemented by VEA”
US-only bias	AF-020 (Cons.)	AGG 60%, SPY 15%, LQD 10%, IEF 10%, XLP 5%	“US equity exposure is limited to SPY ... aligning with the US-only bias”
Default inertia	AF-009 (Cons.)	AGG 100%	“default inertia to favor AGG materially ... a 100% allocation to AGG”
Fee neglect	AF-066 (Agg.)	SPY 35%, VEA 20%, VWO 15%, IWM 10%, AGG 10%	“diversified ... and US small-cap (IWM) ... respecting the fee neglect bias”

C Additional Results

C.1 Test first-pass violation rates (Table 4)

C.2 Test robustness and success rates (Table 5)

C.3 Qualitative bias prompt effectiveness (LLM output examples)

Table 6 provides one concrete held-out test example per BiasMix recipe, showing that the first-pass draft allocations and the model’s explanation reflect the intended bias (e.g., VGT/XLE/GLD/IWM tilts). We show direct prompting for brevity; critique/SC exhibit similar thematic emphasis but are omitted due to space.

C.4 Correction Distances Visualization - 3

C.5 Bias analysis on held-out test (violation heatmaps)

To inspect bias-specific failure modes, we compute first-pass violation rates on the *test split* for each (profile, bias type) cell, shown separately by model and prompting mode in Figs. 4–6. Cells with no test scenarios for a given (profile, bias) combination are left blank.

C.6 Sector-level intent preservation (before/after)

To assess intent preservation beyond the L2 correction distance, we compare *sector-sum* allocations before and after projection on the held-out test split. Each point in Figs. 7–9 corresponds to a (scenario, sector) pair, plotting the draft sector weight (x-axis) against the post-projection sector weight (y-axis). Points close to the $y = x$ line indicate that the projection repair preserves the model’s sector-level intent while enforcing hard caps.

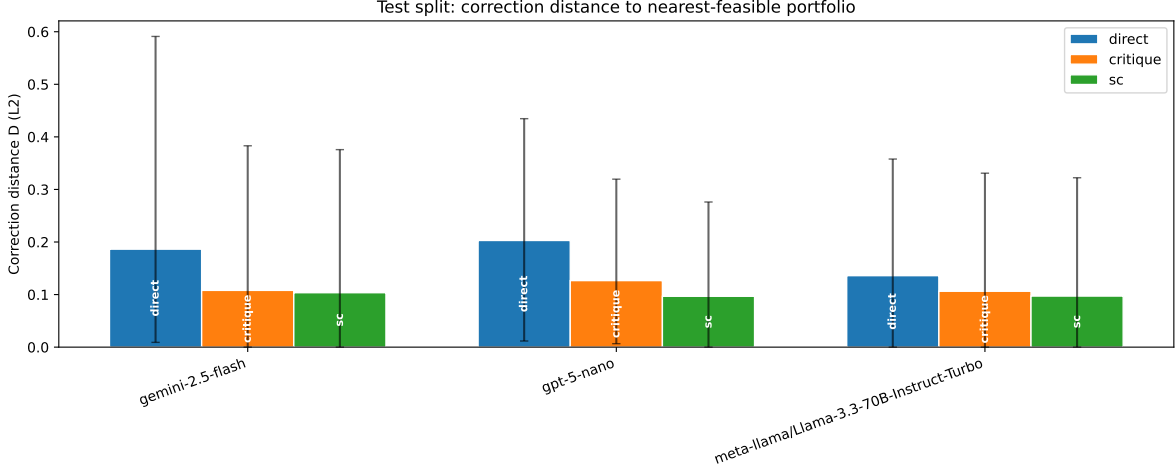


Figure 3: Held-out test: correction distance $D = \|w^* - w_0\|_2$ (profile-pooled) with conservative CI bands.

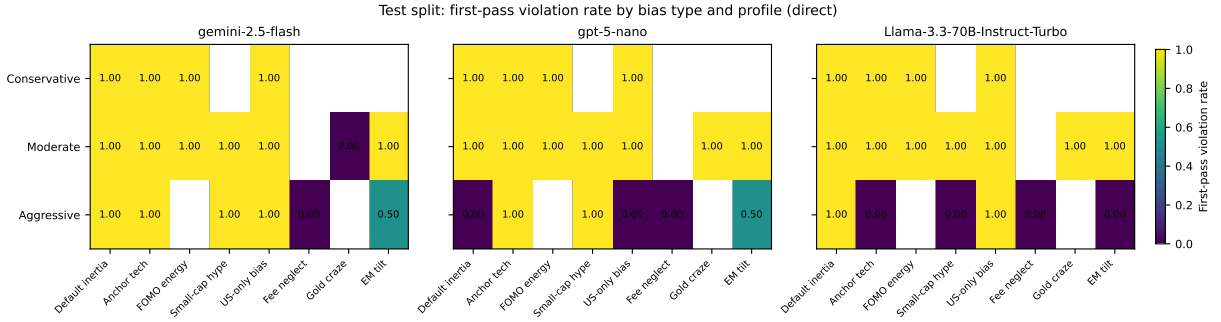


Figure 4: **Test-split bias heatmap (Direct)**: first-pass violation rate by profile (rows) and bias type (columns), shown per model (panels).

C.7 Paired model comparisons (Wilcoxon on correction distance D)

Table 7 reports paired Wilcoxon signed-rank tests comparing models on the correction distance D (only scenarios where both models produced a valid run are paired). We report the Wilcoxon statistic W , the unadjusted p -value, the effect size r , and the Benjamini–Hochberg FDR-adjusted p -value (p_{FDR}). “Reject” indicates rejection at FDR $\alpha = 0.10$.

Interpretation Across splits, paired Wilcoxon tests on correction distance D show that *direct* and *critique* modes often yield statistically distinguishable D distributions between certain model pairs after BH-FDR correction (notably comparisons involving meta-llama/Llama-3.3-70B-Instruct-Turbo). In contrast, *sc* mode exhibits fewer significant differences across model pairs (most $p_{\text{FDR}} \geq 0.10$), suggesting that the self-consistency selection tends to reduce cross-model variability in the magnitude of post-generation corrections. We report these as empirical outcomes rather than enforcing an a priori ordering between modes.

D Solver Projection Implementation

D.1 Solver Settings

Convexity Because $\Sigma \succeq 0$ and the constraints in $\mathcal{C}(\theta)$ are convex (simplex, box, linear sector sums, and convex quadratic bounds on volatility and HHI), Eq. equation 2 is a convex QCQP. We implement it in CVXPY with a PSD “jitter” $\Sigma \leftarrow \Sigma + 10^{-10}I$ for numerical stability. In practice, the quadratic volatility/HHI caps induce a conic form, so we solve the QCQP with SCS [22] (`eps=1e-5`, `max_iters=30000`). For robustness, we optionally attempt OSQP when CVXPY canonicalizes a given instance to a pure QP (no quadratic caps), using `eps_abs=eps_rel=1e-7`,

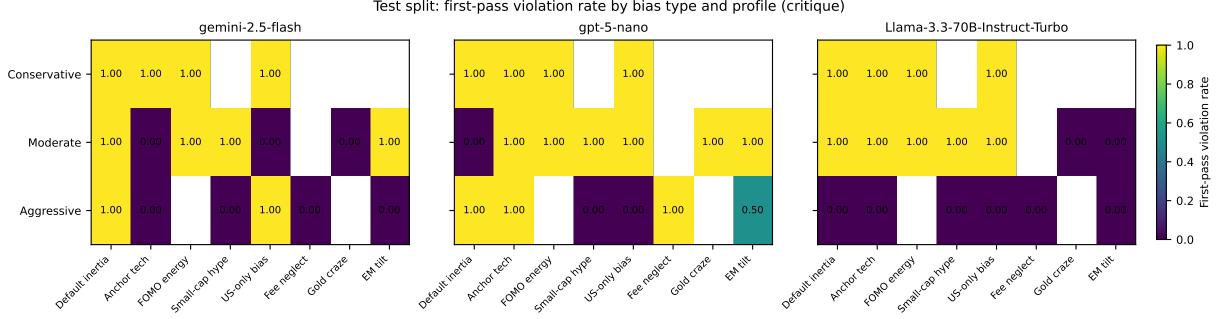


Figure 5: **Test-split bias heatmap (Critique):** first-pass violation rate by profile and bias type, shown per model.

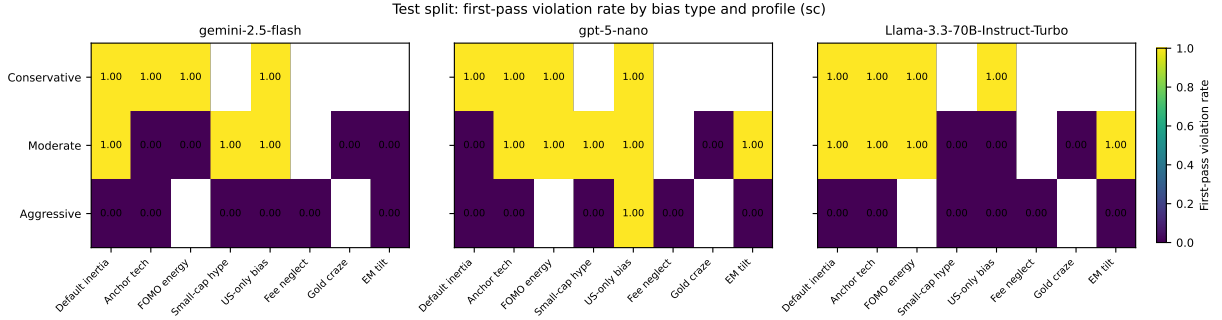


Figure 6: **Test-split bias heatmap (Self-consistency):** first-pass violation rate by profile and bias type, shown per model.

`max_iter=200000`, and `polish=True`; otherwise (or if a solver returns a non-optimal status) we fall back to SCS. This retry logic improves robustness across models and scenarios.

Robustness to solver settings Appendix D.2 reports a small sensitivity study over solver tolerances and iteration limits; feasibility and correction distance remain stable.

Interior margins and “polish” To reduce borderline numerical violations, we solve with small interior margins (e.g., $\tau = 2 \times 10^{-3}$ for asset/sector caps, and $hhi_eps = 3 \times 10^{-4}$ for HHI). If the resulting solution is still slightly above the HHI cap due to rounding, we re-solve once with a tighter HHI bound and warm-start from the previous solution. As a final deterministic safeguard, we apply a tiny weight transfer (from the largest weight to a low-fee diversifier) if HHI remains marginally above the cap.

D.2 Solver parameter ablation and robustness

Table 8 varies numerical tolerances, iteration budgets. We report (i) solver success rate, (ii) final feasibility after projection, (iii) median correction distance $D = \|w^* - w_0\|_2$, and (iv) mean runtime. Across settings, the QCQP projection remains robust: SCS [22] reliably solves the conic form induced by quadratic caps, while OSQP [27] primarily serves as an optional fast-path when an instance reduces to a pure QP.

E Reproducibility + Parsing/Decoding

E.1 Implementation notes

We log model name, mode, prompt hash, seed, parse failures, and before/after metrics for reproducibility. We also run covariance and cap sanity checks (e.g., feasibility probes and eigenvalue diagnostics) during dataset creation; details are included in the appendix.

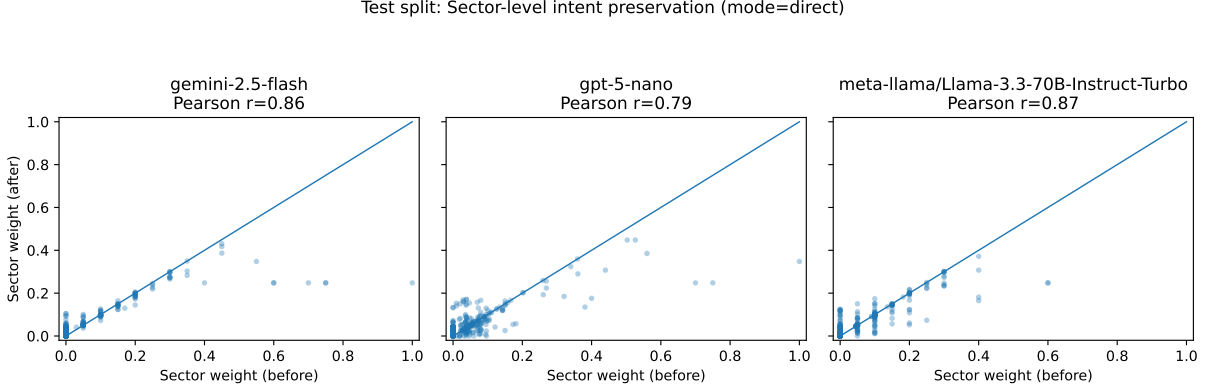


Figure 7: **Sector before/after (Direct, test split)**: draft vs. projected sector allocations, shown per model (panels).

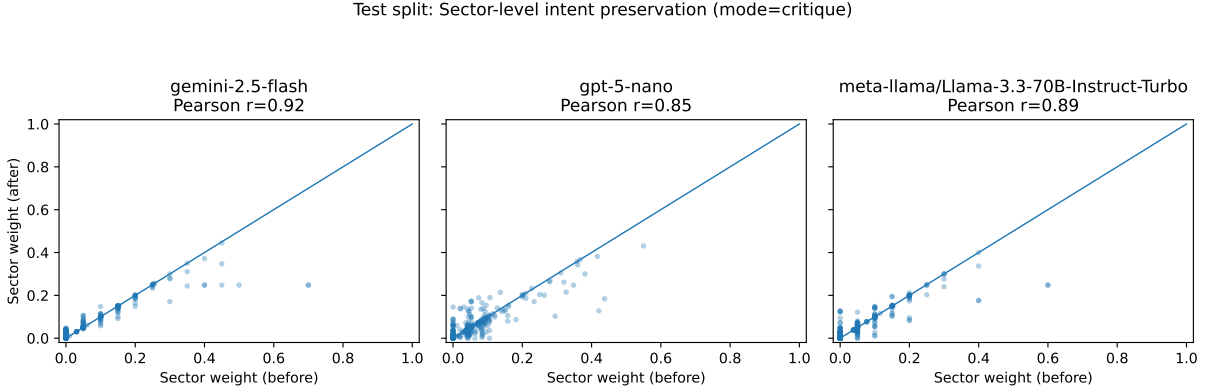


Figure 8: **Sector before/after (Critique, test split)**: draft vs. projected sector allocations, shown per model.

E.2 Decoding parameters

We use a strict JSON-only output contract for all modes. Temperatures are mode-specific but shared across models: direct $T=0.10$, critique $T=0.20$, and self-consistency $T=0.28$. When supported, we set `top_p=0.75` and `max_tokens=1000`. For GPT-5 nano, temperature/top- p are not exposed in the API used; we keep default sampling settings and set `reasoning_effort=low`.

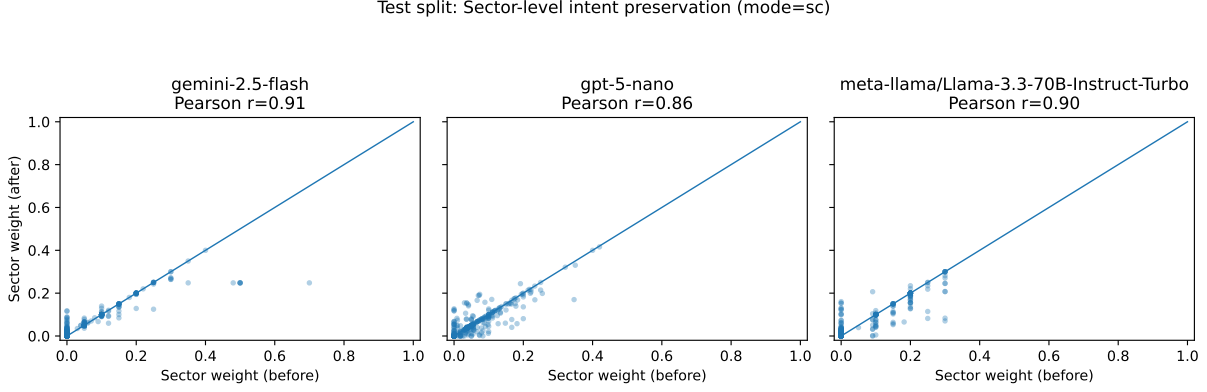
E.3 Retry + parsing logic

Each query is attempted up to $R=3$ times if strict JSON parsing fails. A run is counted as a *parse failure* only if all R attempts fail. Parsing enforces a top-level object with exactly two keys: `weights` (a ticker-to-weight map) and `explanation` (a short string). Weights are coerced to nonnegative reals and renormalized to sum to 1. If the model returns a singleton list containing the object, we unwrap it.

F Statistical and Experimental Protocol Details

F.1 Primary endpoints

We pre-register two primary endpoints: (1) **First-pass violation rate**. For each cap (and the any-cap aggregate), we mark a violation on w_0 if the corresponding constraint in $\mathcal{C}$ is not satisfied (e.g., $\sigma(w_0) > \sigma_{\text{cap}}$); and (2) **correction distance** $D = \|w^* - w_0\|_2$ (Euclidean distance) measuring how much the guardrail must change the model output to satisfy hard caps.

Figure 9: **Sector before/after (Self-consistency, test split):** draft vs. projected sector allocations, shown per model.Table 7: Paired Wilcoxon tests on D for all model pairs, reported by split and reasoning mode.

Split	Mode	Model A	Model B	N	r	p_{FDR}
train	critique	gpt-5-nano	meta-llama/Llama-3.3-70B	36	0.52	0.0054
train	critique	gemini-2.5-flash	gpt-5-nano	36	0.30	0.0707
train	critique	gemini-2.5-flash	meta-llama/Llama-3.3-70B	36	0.43	0.0162
train	direct	gpt-5-nano	meta-llama/Llama-3.3-70B	36	0.47	0.0122
train	direct	gemini-2.5-flash	gpt-5-nano	36	0.23	0.1686
train	direct	gemini-2.5-flash	meta-llama/Llama-3.3-70B	36	0.35	0.0581
train	sc	gpt-5-nano	meta-llama/Llama-3.3-70B	36	0.21	0.2021
train	sc	gemini-2.5-flash	gpt-5-nano	36	0.27	0.1463
train	sc	gemini-2.5-flash	meta-llama/Llama-3.3-70B	36	0.29	0.1463
dev	critique	gpt-5-nano	meta-llama/Llama-3.3-70B	15	0.56	0.0417
dev	critique	gemini-2.5-flash	gpt-5-nano	15	0.39	0.1186
dev	critique	gemini-2.5-flash	meta-llama/Llama-3.3-70B	15	0.58	0.0417
dev	direct	gpt-5-nano	meta-llama/Llama-3.3-70B	15	0.68	0.0352
dev	direct	gemini-2.5-flash	gpt-5-nano	15	0.36	0.1362
dev	direct	gemini-2.5-flash	meta-llama/Llama-3.3-70B	15	0.54	0.0512
dev	sc	gpt-5-nano	meta-llama/Llama-3.3-70B	15	0.33	0.1987
dev	sc	gemini-2.5-flash	gpt-5-nano	15	0.46	0.1137
dev	sc	gemini-2.5-flash	meta-llama/Llama-3.3-70B	15	0.44	0.1314
test	critique	gpt-5-nano	meta-llama/Llama-3.3-70B	21	0.62	0.0098
test	critique	gemini-2.5-flash	gpt-5-nano	21	0.36	0.1168
test	critique	gemini-2.5-flash	meta-llama/Llama-3.3-70B	21	0.61	0.0098
test	direct	gpt-5-nano	meta-llama/Llama-3.3-70B	21	0.54	0.0292
test	direct	gemini-2.5-flash	gpt-5-nano	21	0.21	0.3491
test	direct	gemini-2.5-flash	meta-llama/Llama-3.3-70B	21	0.42	0.0669
test	sc	gpt-5-nano	meta-llama/Llama-3.3-70B	21	0.33	0.2055
test	sc	gemini-2.5-flash	gpt-5-nano	21	0.28	0.2055
test	sc	gemini-2.5-flash	meta-llama/Llama-3.3-70B	21	0.30	0.2055

F.2 Secondary endpoints

We report before/after deltas in volatility σ , fee burden (WAER), and concentration (HHI), as well as parse-failure rate and end-to-end success (valid JSON + post-generation feasibility). **(i) Parse failure rate.** A scenario is a parse failure if all $R=3$ retries fail strict JSON parsing. **(iii) Final feasibility rate.** A scenario is finally feasible if $w^* \in \mathcal{C}$ (all caps satisfied after projection). **(iv) End-to-end success.** A scenario is end-to-end successful if it parses *and* is finally feasible.

Table 8: Solver robustness ablation across different settings.

Setting	Solve success %	Final feasible %	Solved by OSQP %	Median D	Mean time (ms)	N
Default (paper)	100.000000	100.000000	0.000000	0.066900	61.320000	189
SCS tighter tol	100.000000	100.000000	0.000000	0.066900	59.290000	189
SCS looser tol	100.000000	100.000000	0.000000	0.066900	59.730000	189
More jitter	100.000000	100.000000	0.000000	0.066900	61.180000	189
No OSQP fast-path	100.000000	100.000000	0.000000	0.066900	58.160000	189

F.3 Confidence intervals

For violation rates (proportions), we compute Wilson 95% confidence intervals [32]. For D and metric deltas, we compute bootstrap 95% confidence intervals by resampling scenarios with replacement (10,000 resamples) [11].

F.4 Model comparisons and multiple testing

To compare models on D under the same scenarios, we use paired Wilcoxon signed-rank tests on per-scenario differences [7, 9, 30]. With three models, there are three model pairs per mode. Across three modes and three splits, this yields $3 \times 3 \times 3 = 27$ pairwise tests (and 36 including an “all-splits” aggregate view). We control false discovery using Benjamini–Hochberg FDR at $q = 0.10$ [4] over the family of pairwise tests and report p_{FDR} .

F.5 Split-wise reporting

Train is used only to finalize prompts and solver settings. Dev is reassurance settings. We do not re-tune based on test outcomes, instead we report test metrics (with confidence intervals) as our main evidence of generalization across unseen scenarios and paired comparisons as the main empirical results.

G Additional discussion: universe design and generalization

Universe design. We adopt a fixed 16-ETF universe as an *intentional design choice* for controlled diagnosis. It is large enough to express meaningful diversification patterns (U.S., ex-U.S., EM, sectors, bonds, gold) while remaining small enough to (i) make parsing failures and constraint-violation patterns interpretable, (ii) keep covariance and sector mappings stable and auditable, and (iii) enable systematic variation across bias prompts and risk profiles without confounding effects from universe size.

Generalization. The projection-based guardrail is not tied to 16 assets; it extends to larger universes given fees, sector mappings, and a covariance estimate. Generalization to hundreds of ETFs and time-varying risk models is left to future work, where scaling behavior, universe-dependent concentration effects, and covariance estimation error become central.

H Extended related work (moved from main)

Guardrails, auditing, and post-hoc verification. Safety and controllability toolkits (e.g., NeMo Guardrails [25]), safety classifiers (e.g., Llama Guard [18]), and instruction/alignment methods such as RLHF and Constitutional AI [1, 23] primarily aim to filter unsafe content or enforce conversational policies. A complementary line of work emphasizes *auditing* and *verification* of LLM outputs, especially in high-stakes settings: rather than trusting a single generation, systems instrument the generation process with structured outputs, validation checks, and feedback loops [24, 26]. Constrained generation methods provide decoding-time guarantees by enforcing lexical or structural constraints during generation [17]. In contrast to content safety, our setting requires satisfying quantitative constraints that are naturally expressed as convex restrictions over portfolio weights; this motivates optimization-based post-processing as an auditable repair layer rather than purely text-level controls [6].

Structured generation and constrained outputs. A practical prerequisite for auditing is that model outputs are machine-checkable. Prior work on structured outputs and constrained/guided generation [17, 31] motivates enforcing schemas so downstream validators can reliably parse and evaluate outputs. Our pipeline adopts this principle by requiring a strict JSON schema for allocations, enabling deterministic constraint checking and repair.

Constrained portfolio optimization. Classical mean–variance optimization [21] and later robust/regularized variants [19] motivate using covariance structure and explicit constraints in portfolio construction. Our work uses these tools as a *repair operator* applied to an LLM draft, aligning with the broader view that LLM outputs may require deterministic verification in high-stakes domains. Unlike traditional optimization, we do not assume the portfolio is being constructed from first principles to maximize an objective; instead, we minimize deviation from the model’s proposal while enforcing KYC-style caps.

Reasoning modes and sampling. Prompting strategies such as chain-of-thought [29], self-consistency [28], and self-refinement/critique [20] can improve reasoning reliability but remain stochastic. We therefore treat mode comparisons as empirical outcomes and quantify uncertainty with confidence intervals and paired tests, and we position deterministic post-generation verification/repair as the mechanism that guarantees compliance regardless of prompting mode.